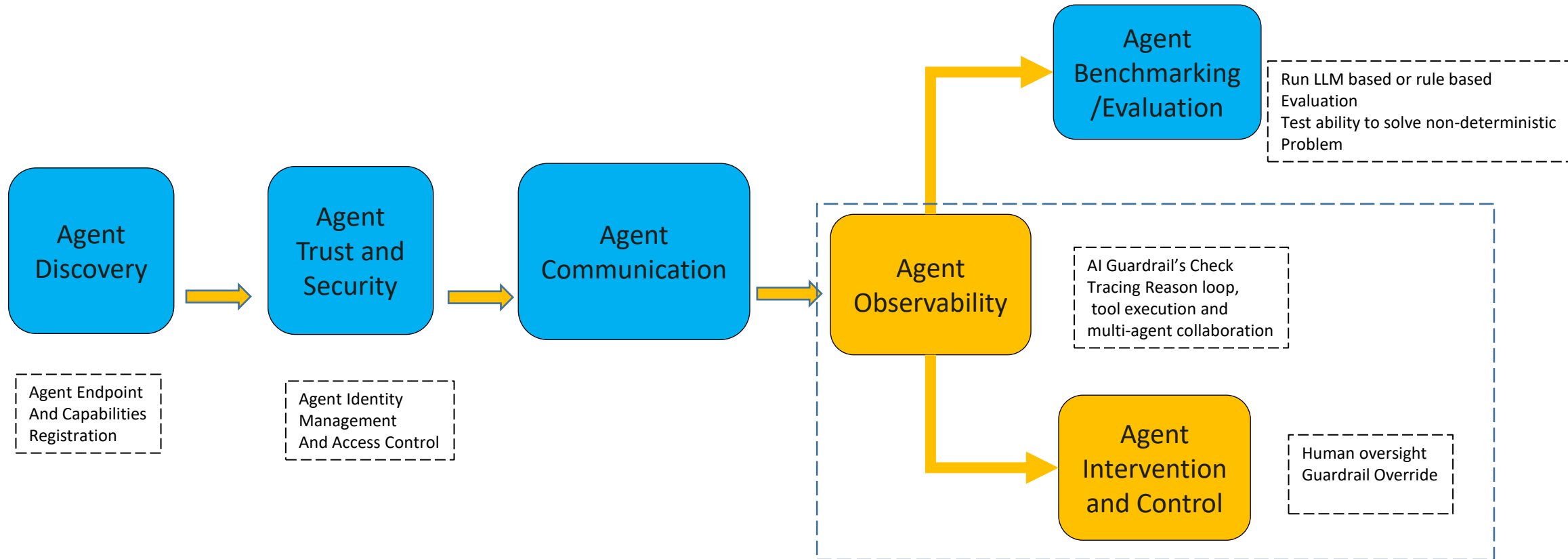


Problem Statement for Network Agent Observability, Intervention and Control

IETF 126 22/07/2026

Qin Wu, Daniele Ceccarelli, ZhengQiang Li, Luis M.
Contreras , Qiufang Ma

Autonomous Agent Operational Lifecycle



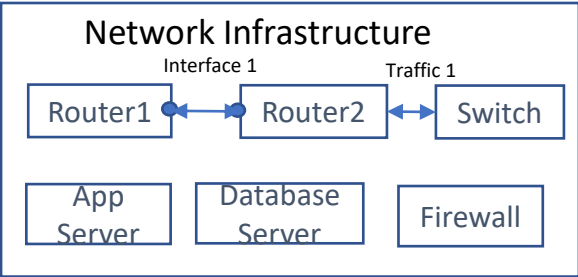
Why Network Agent Observability, Intervention, Control?

- We always believe AI for good
 - UC Cases: service complaint Handling, IP+Optical cross layer cross domain fault management, etc [I-D.king-rokui-ainetops-usecases].
 - Value: OPEX/CAPEX reduction, ROI service (Service Manager), site visit reduction (Field Engineer), etc.
- What if AI Agent goes wrong (e.g., misbehavior, intent/goal deviation)
 - **Non-Deterministic Execution Paths**
 - same input with different chains of tool calls, retries, and reasoning steps
 - **Cascading Failures That Look Like Success**
 - An agent can return a confident-sounding wrong answer after silently failing to retrieve context
 - **Hidden Cost Accumulation**
 - single user query might trigger 15 LLM calls across different models
 - agent loop can burn through a monthly token budget in hours
 - **Extreme Latency Variance**
 - Same query might complete in 800ms or 30 seconds depending on the reasoning path

Network Agent Observability vs Network Observability



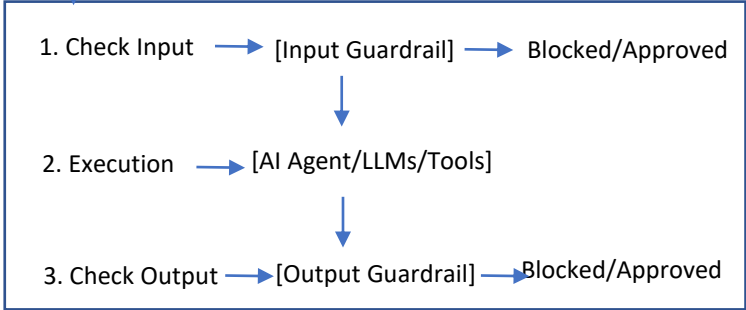
Network Observability



Traffic Metrics:
Throughput, Packet loss, Latency, Jitter, RTT, etc
Host Metrics: CPU Usage, Memory Usage, etc
Network Topology:
L3 Topo, L2 Topo, TE Topo
Telemetry Protocols:
YANG Push, Netflow, sFlow, eBPF, Opentelemetry

User Input

Agent Observability



- gen_ai.request.model: "gpt-4o"
- gen_ai.usage.input_tokens: 450
- gen_ai.response.completion: "..."
...

Final Output

Metric related to Reasoning, Decision Loop, Generation Quality:
Tool Usage, Token usage, Model Cost, Guardrail failure, etc

Protocol: OpenTelemetry+AI Guardrail

Network Agent Observability and Network Operation

- AI Agent for Network Operation
 - Agentic Network Architecture with multi-scenario autonomy and multi-agent collaboration across layer, across Domain
 - See reference architecture `draft-wmz-nmrg-agent-ndt-arch`
 - Digital Twin Agent for real time emulation and run what if scenarios
 - Fault Management Agent for intelligent anomaly and Threat Detection, Automated Root Cause Analysis
 - Quality Optimization Agent dynamically to adjust configurations or alert thresholds
- Network Protocol impacts on Agent Observability
 - API and Tool Chaining Tracking needs to establish dependency between Network latency, packet loss and Agent reasoning chain
 - Network trace links system metrics such as HTTP errors with Agent specific behavior such as prompt execution
 - In case of Agent to Agent, Agent to tool, Agent to Human interaction
 - Network Observability ensure Agent multi-step workflow without data leakage through unencrypted channel

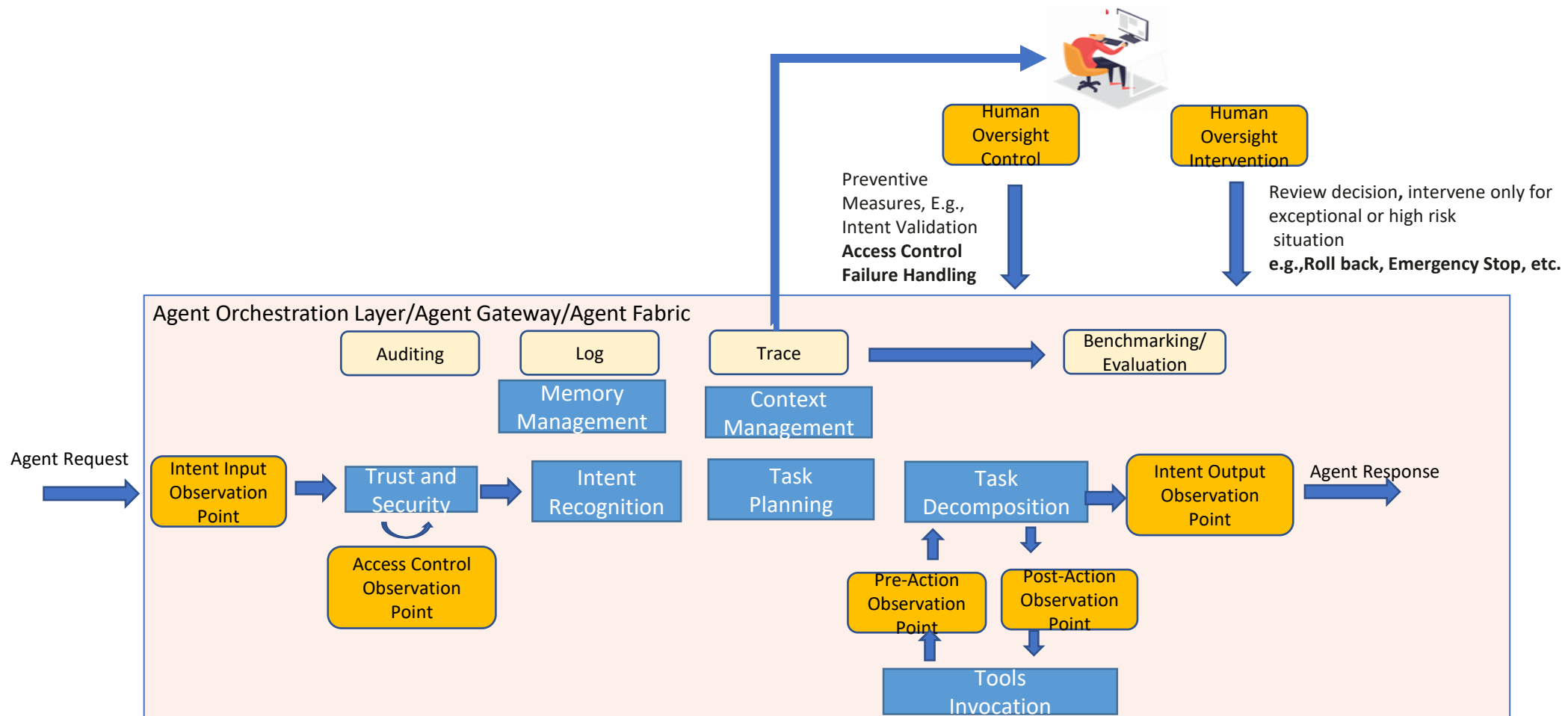
What Metrics are needed for Network Agent Observability

Category	Metric Name	Metric Description
Token Metrics (Cost & Volume)	Input/Prompt Tokens	A counter tracking the absolute volume of text sent to the model
	Output/Completion Tokens	A counter tracking how much data the model generated
	Cached Tokens	counter monitoring how many input tokens hit context caches
	Estimated Inference Cost	An aggregated financial metric calculating dollar spent based on token counts mapped to model provider price tiers.
Operational & Latency Metrics (Performance)	Time-to-First-Token	histogram measuring the duration between sending a prompt and receiving the first streamed response chunk
	Execution Latency	total time taken for an LLM response, a Vector DB query, or a Tool call to complete
	Relevance Retrieval Latency	Tracks how long it takes your Vector Database (e.g., Pinecone, Chroma) to fetch semantic contexts
Agent Lifecycle & Workflow Metrics (Behavior)	Agent Turn / Hop Count	counter tracking how many loops an agent performs (e.g., Thought → Action → Observation) before providing a final response
	Tool Invocation Frequency	how often specific functions or APIs are chosen by the agent
	Task Success vs. Failure Rate	Tracks whether an agent completed its objective or exited due to errors, depth limits, or guardrail intervention
	Agent Handoff Count	In multi-agent frameworks (like CrewAI or LangGraph), this tracks how many times a task is delegated from one specialized agent to another.
Guardrail & Quality Metrics (Safety & Alignment)	Policy Breach Counters	Aggregates how often specific guardrails are triggered (e.g., prompt injection, PII leakage, or toxic outputs)
	Online Evaluation Scores	Numeric values mapping automated judges (e.g., hallucination indexes or semantic similarity scores) over time

Challenges of Network Agent Observability

- Unable to provide continuous monitoring of an AI system's internal logic or its long-running execution paths
 - Lack standardized health metrics to assess Agent misbehavior
 - e.g., agent's reasoning logic score
 - internal confidence levels.
 - Limited Transparency in Planning and Decision-Making
 - Unable to validate the safety or intent of an agent's planned mutations
 - Hard to attribute accountability to a specific agent entity or human in case of
 - System failures
 - Performance degradations
 - Or unintended consequences occur
 - Unable to match the speed and scale of the network management Agent applications

AI Guardrail Example for Agent Intervention and Control



Rule based filter: Pattern Matching, regular expression, Keywords Blocking
LLM based safety Classifier: LLM as a judge

Guardrail Libraries: Policy Language to express Policy
Prompt Engineering Constraint

Open source projects related to Agent Control and Intervention

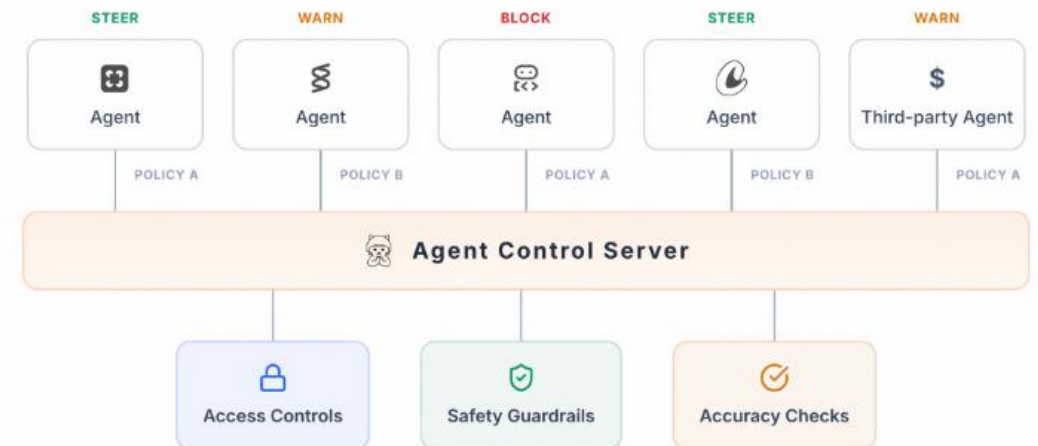
- A2H (agent to human) protocol:
 - <https://github.com/twilio-labs/Agent2Human>
- Agent control:
 - <https://github.com/agentcontrol/agent-control>

What A2H enables

A2H defines five core intent types that cover the essential interactions between agents and humans:

Intent	Purpose	Example
INFORM	One-way notification	"Your order has shipped"
COLLECT	Gather structured data	"Please provide your shipping address"
AUTHORIZE	Request approval with authentication	"Approve this \$500 transaction"
ESCALATE	Hand off to human support	"Connecting you with a live agent"
RESULT	Report task completion	"Your refund has been processed"

Unidirectional interaction (Agent->Human)



Challenges on Network Agent Intervention and Control

- Static measures are proving insufficient for the full operational lifecycle
 - Lack of human oversight and human-AI semantic intent exchange hinder timely risk mitigation
 - E.g., manual human review for edge cases or escalations
- no standardized protocols for intervention and recovery mechanisms.
 - e.g., reverting to a last known safe state or undoing a series of autonomous actions that introduce substantial operational risk
 - Fall short for multi-step execution

What needs to be standardized

- Network Agent Observability, Intervention and Control Network Management Architecture:
 - Enforcing boundaries, detecting, evaluating, interrupting, correcting, and recovering from agent behavior
 - within operational and compliance boundaries.
- Network Agent Observability protocol Enabling Agent behavioral assessment through analysis of observed operational data (logs, metrics, traces, etc.)
- Human and Agent Interaction protocol for Human Escalation/ Intervention, Agent Intervention and Control